

**SIMATS ENGINEERING
ASSESSMENT TOOL 1**

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1709

CO1-AT1-Analytical Problem Solving

Register Number	192212182
Name	U Fayaz Ahmad

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)

Assessment Tool Weightage: 40%

QUESTIONS: 5

Total Marks: 50

1. Solve the Water Jug problem: you are given 2 jugs, a 4-gallon one and 3-gallon one. Neither has any measuring maker on it. There is a pump that can be used to fill the jugs with water. How can you get exactly 2 gallons of water into 4-gallon jug? Explicit assumptions: A jug can be filled from the pump, water can be poured out of a jug onto the ground, water can be poured from one jug to another and that there are no other measuring devices available.

Code

Water Jug Problem (4 Gallon & 3 Gallon)

```
def water_jug():  
    jug4 = 0  
    jug3 = 0  
  
    print("Initial State:", (jug4, jug3))  
  
    jug3 = 3  
    print("Fill 3-gallon:", (jug4, jug3))  
  
    jug4 = jug3  
    jug3 = 0  
    print("Pour into 4-gallon:", (jug4, jug3))  
  
    jug3 = 3  
    print("Fill 3-gallon again:", (jug4, jug3))
```

```
transfer = 4 - jug4
jug4 = 4
jug3 -= transfer
print("Fill 4-gallon:", (jug4, jug3))
```

```
jug4 = 0
print("Empty 4-gallon:", (jug4, jug3))
```

```
jug4 = jug3
jug3 = 0
print("Final State:", (jug4, jug3))
```

water_jug()

The screenshot shows a web browser window with the Programiz Python Online Compiler. The code in the editor is as follows:

```
1 # Water Jug Problem (4 Gallon & 3 Gallon)
2
3 def water_jug():
4     jug4 = 0
5     jug3 = 0
6
7     print("Initial State:", (jug4, jug3))
8
9     jug3 = 3
10    print("Fill 3-gallon:", (jug4, jug3))
11
12    jug4 = jug3
13    jug3 = 0
14    print("Pour into 4-gallon:", (jug4, jug3))
15
16    jug3 = 3
17    print("Fill 3-gallon again:", (jug4, jug3))
18
19    transfer = 4 - jug4
20    jug4 = 4
21    jug3 -= transfer
22    print("Fill 4-gallon:", (jug4, jug3))
```

The output window displays the following execution results:

```
Initial State: (0, 0)
Fill 3-gallon: (0, 3)
Pour into 4-gallon: (3, 0)
Fill 3-gallon again: (3, 3)
Fill 4-gallon: (4, 2)
Empty 4-gallon: (0, 2)
Final State: (2, 0)

=== Code Execution Successful ===
```

The browser's taskbar at the bottom shows the Windows Start button, a search bar, and various application icons. The system clock indicates 11:31 PM on 16-07-2026.

2. Find out how Mars Rover, analyse and explore the samples on Mars by answering the following questions.
- i. What are the percept's for this agent?
 - ii. Characterize the operating environment.
 - iii. What are the actions the agent can take?
 - iv. How can one evaluate the performance of the agent?
 - v. What sort of agent architecture do you think is most suitable for this agent? Why?

Code

```
class MarsRover:

    def perceive(self):
        print("Perceiving environment...")
        print("Rock detected")
        print("Obstacle detected")

    def move(self):
        print("Moving Forward")

    def collect_sample(self):
        print("Collecting Soil Sample")

    def analyze(self):
        print("Analyzing Sample")

    def send_data(self):
        print("Sending Data to Earth")

rover = MarsRover()

rover.perceive()
rover.move()
rover.collect_sample()
rover.analyze()
rover.send_data()
```



- Explore the search space using search strategies and formulate the problem components for the 8-Queens problem using the following information: Place 8 queens on a chessboard such that none of the queen's attack any of the others. A configuration of 8 queens on the board as shown in the figure below, but this does not represent a solution as the queen in the first column is on the same diagonal as the queen in the last column.

Code

N = 8

board = [[0]*N for _ in range(N)]

def is_safe(board,row,col):

for i in range(col):
if board[row][i]:
return False

i=row
j=col
while i>=0 and j>=0:
if board[i][j]:
return False
i-=1
j-=1

i=row
j=col
while i<N and j>=0:
if board[i][j]:

```

        return False
        i+=1
        j-=1

    return True

def solve(board,col):

    if col>=N:
        return True

    for i in range(N):

        if is_safe(board,i,col):

            board[i][col]=1

            if solve(board,col+1):
                return True

            board[i][col]=0

    return False

solve(board,0)

for row in board:
    print(row)

```

The screenshot displays the Programiz Python Online Compiler interface. The code editor on the left contains a Python script for solving the N-Queens problem. The script defines a function `is_safe` (partially visible) and a recursive `solve` function. The `solve` function iterates through rows, checking for safe positions for a queen. Once a solution is found, it prints the board configuration. The output panel on the right shows the resulting 8x8 board, where each row contains exactly one queen (represented by '1') and all other cells are '0'. The board configuration is as follows:

```

[1, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 1, 0]
[0, 0, 0, 1, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 1]
[0, 1, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 1, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 1, 0, 0]
[0, 0, 1, 0, 0, 0, 0, 0]

```

Below the board configuration, the output panel indicates "Code Execution Successful". The browser's address bar shows the URL `programiz.com/python-programming/online-compiler/`. The bottom of the image shows a Windows taskbar with the time 11:35 PM on 16-07-2026.

4. A customer wants to travel from the one location to another location using OLA Cab booking mobile application. The customer can select the any of the cab type as mini, micro, sedan, shared, prime and so on based on his comfort to reach the destination. Identify what type of problem-solving agent can be used and write the pseudocode for the above problem.

Code

```
source = input("Enter Source: ")  
destination = input("Enter Destination: ")
```

```
print("\nCab Types")  
print("1. Mini")  
print("2. Micro")  
print("3. Sedan")  
print("4. Prime")  
print("5. Shared")
```

```
choice = int(input("Choose Cab: "))
```

```
cab = {  
    1:"Mini",  
    2:"Micro",  
    3:"Sedan",  
    4:"Prime",  
    5:"Shared"  
}
```

```
print("\nBooking Successful")  
print("Cab Type :",cab[choice])  
print("From :",source)  
print("To :",destination)  
print("Thank You!")
```

New File at / - fazazahmad...AI Problem SolvingWelcome to Colab - ColabCopy of Welcome to Colab - Online Python Compiler (Int...Ask Gemini

programiz.com/python-programming/online-compiler/Ask Google

ProgramizPython Online Compiler

Trade Global Indices

Start Trading

Programiz PRO >

main.py

121:"Mini",

132:"Micro",

143:"Sedan",

154:"Prime",

165:"Shared"

17}

18

19while True:

20try:

21choice = int(input("Choose Cab: "))

22if choice in cab:

23break

24else:

25print("Invalid choice. Please choose a number between 1 and 5.")

26except ValueError:

27print("Invalid input. Please enter a number.")

28

29print("\nBooking Successful")

30print("Cab Type :",cab[choice])

31print("From :",source)

32print("To :",destination)

33print("Thank You!")

Run

Output

Clear

Enter Source: chennai

Enter Destination: nellore

Cab Types

1. Mini

2. Micro

3. Sedan

4. Prime

5. Shared

Choose Cab: 4

Booking Successful

Cab Type : Prime

From : chennai

To : nellore

Thank You!

=== Code Execution Successful ===

KHAI MỞ KẾT NỐI TOÀN CẦU

Adelaide University

TÌM HIỂU THÊM

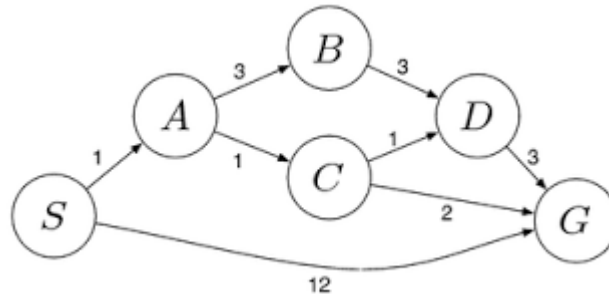
Search

ENG IN

11:39 PM 16-07-2026

5. A logistics company operates a delivery network where packages must be transported between warehouses at minimum cost. Each route between warehouses has an associated travel cost (fuel + time). The company wants to determine the least-cost path from the starting warehouse S to the destination warehouse G using the Uniform Cost Search (UCS) algorithm.

The delivery network is represented as a weighted graph:



Code

```
import heapq
```

```
graph = {
    'S': [('A',1),('G',12)],
    'A': [('B',3),('C',1)],
    'B': [('D',3)],
    'C': [('D',1),('G',2)],
    'D': [('G',3)],
    'G': []
}
```

```
def ucs(start,goal):
```

```
    pq=[(0,start,[])]
```

```
    visited=set()
```

```
    while pq:
```

```
        cost,node,path=heapq.heappop(pq)
```

```
        if node in visited:
            continue
```

```
        visited.add(node)
```

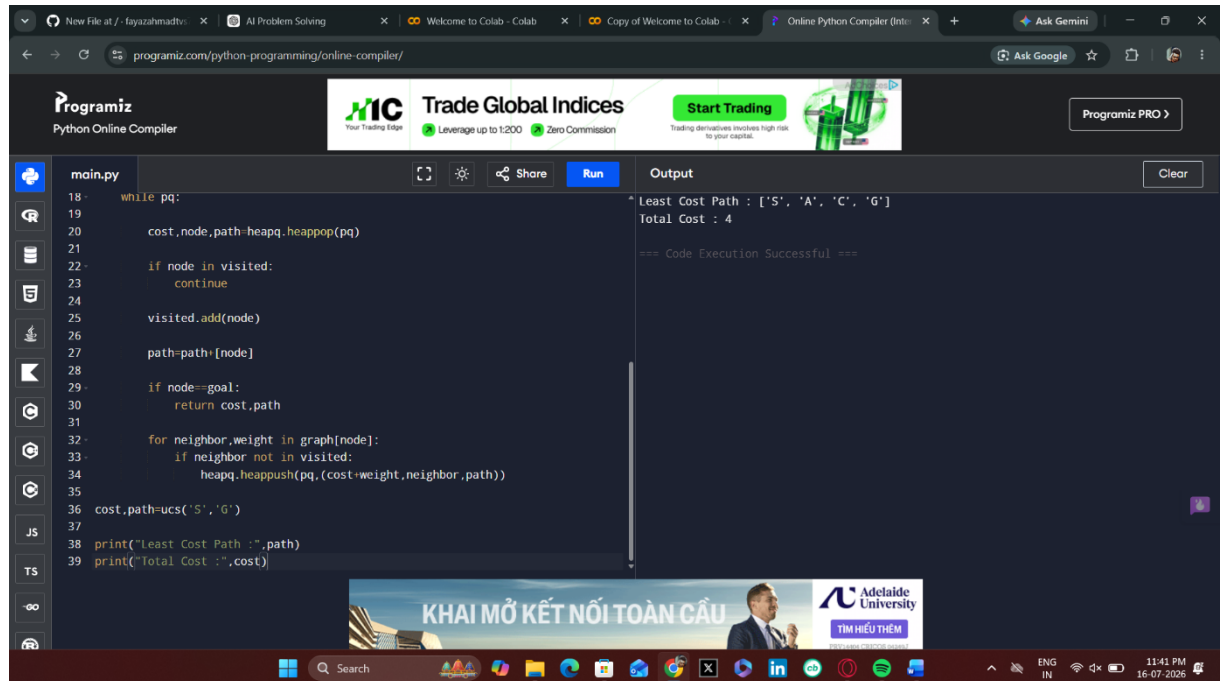
```
        path=path+[node]
```

```
        if node==goal:
            return cost,path
```

```
for neighbor,weight in graph[node]:  
    if neighbor not in visited:  
        heapq.heappush(pq,(cost+weight,neighbor,path))
```

```
cost,path=ucs('S','G')
```

```
print("Least Cost Path :",path)  
print("Total Cost :",cost)
```



The screenshot displays a web browser window with the Programiz Python Online Compiler interface. The browser's address bar shows the URL `programiz.com/python-programming/online-compiler/`. The page header includes the Programiz logo and several advertisements, such as "Trade Global Indices" and "Start Trading". The main content area is divided into a code editor on the left and an output panel on the right. The code editor shows a Python script for finding the least cost path using a priority queue. The output panel displays the results of the code execution: "Least Cost Path : ['S', 'A', 'C', 'G']" and "Total Cost : 4". The browser's taskbar at the bottom shows various application icons and the system clock, which indicates the time is 11:41 PM on 16-07-2026.

```
18: while pq:  
19:  
20:     cost,node,path=heapq.heappop(pq)  
21:  
22:     if node in visited:  
23:         continue  
24:  
25:     visited.add(node)  
26:  
27:     path=path+[node]  
28:  
29:     if node==goal:  
30:         return cost,path  
31:  
32:     for neighbor,weight in graph[node]:  
33:         if neighbor not in visited:  
34:             heapq.heappush(pq,(cost+weight,neighbor,path))  
35:  
36: cost,path=ucs('S','G')  
37:  
38: print("Least Cost Path :",path)  
39: print("Total Cost :",cost)
```

Least Cost Path : ['S', 'A', 'C', 'G']
Total Cost : 4
=== Code Execution Successful ===

Code of Conduct:

I _____U Fayaz Ahmad_____192212182_____ (Name / Reg. No.) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criterion	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Max Marks
Problem Understanding	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem	10
Method Selection	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method	10
Solution Process	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process	12.5
Accuracy of Calculation	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations	10
Interpretation of Results	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation	7.5

